A

SYNOPSIS REPORT

On

"PriceSense – Predict. Price. Approve"

Submitted in partial fulfilment of the requirements of the degree of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING



Submitted by

Srijug Trehan (11023210165)

Harsith Chandrasekaran (11023210168)

Ayush Trivedi (11023210170)

Under Supervision of

Mr. Vikas Deswal

Assistant Professor

Department of Computer Science & Engineering

Faculty of Engineering and Technology

SRM University Delhi-NCR

39, Rajiv Gandhi Education City, Sonepat, Haryana – 131029

MAY 2026

2. Problem Statement

E-commerce sellers must continuously decide how much stock to hold and what price to charge. Demand shifts with seasonality, offers, customer interest, product trends and regional preferences. Small and niche online sellers usually take these decisions manually from basic sales reports and spreadsheets, which leads to stockouts, excess inventory, lost sales and reduced profit. Pricing and inventory tools often work in isolation, and customer sentiment is rarely considered.

Problem statement: *How can an AI-based system forecast future demand and recommend suitable prices and stock actions for niche e-commerce products, while keeping final decisions under human (manager) control?*

3. Objective and Scope of the Project

3.1 Objectives

1. Forecast product demand for the next 7 and 30 days from historical sales, price, discount, promotion and seasonal data.
2. Compare a baseline method against machine-learning models (XGBoost/LightGBM, with LSTM as an optional extension) using MAE, RMSE and MAPE.
3. Detect stockout and excess-inventory risk by comparing forecast demand with available stock and reorder thresholds.
4. Analyse product-review or social-media text and generate a sentiment score using VADER.
5. Recommend a price that respects business constraints (approved minimum and maximum price, cost floor).
6. Provide an interactive dashboard where the manager reviews the forecast, risk alerts, sentiment, recommended price and a plain-language explanation, then approves or rejects the change.

3.2 Scope

- Focus on one niche category (for example phone accessories, skincare or fashion products) using historical or clearly labelled simulated data.
- Inputs: sales, inventory, product cost, selling price, promotions and review text.
- Outputs: demand forecast, stock alert, sentiment score, recommended price and explanation.
- Price changes are never applied automatically; the manager has final approval.
- Out of scope: live marketplace integration, payment processing and fully automated repricing.

3.3 Expected Outcome and Benefit to the End User

A working prototype that turns raw sales, stock and review data into actionable, explainable recommendations. Small sellers gain a simple analytics tool that reduces manual effort, lowers stockout and overstock risk, and supports data-driven pricing.

4. Methodology

The system follows an eight-stage pipeline (see the system architecture in Section 4.2).

1. **Data collection and preparation:** Collect or generate product, pricing, sales, inventory and review-text data, clean it and load it into MySQL.
2. **Baseline forecast:** Use a simple method (for example moving average or seasonal naive) as a comparison point.

3. **Machine-learning forecast:** Train XGBoost or LightGBM on lag sales, selling price, discount, promotion flag, stock level and seasonal indicators. An LSTM model may be added for comparison.
4. **Model evaluation:** Compare models with MAE, RMSE and MAPE using time-ordered (roll-forward) validation to avoid data leakage.
5. **Sentiment analysis:** Apply VADER to review or social-media text to obtain a sentiment score per product.
6. **Inventory-risk analysis:** Compare forecast demand against stock and reorder thresholds to flag stockout or excess-stock risk.
7. **Constrained price recommendation:** Estimate price elasticity, compute a candidate price from predicted demand, cost, inventory and sentiment, then clip it to the approved minimum and maximum range.
8. **Manager approval:** Show the recommendation and its explanation on the dashboard; the manager approves, rejects or adjusts it. An LLM-based assistant may be added to explain recommendations in natural language.

4.1 Process Flow

1. **Data sources (CSV files):** product and pricing, sales, inventory and review text are loaded into MySQL.
2. **Flask REST API and preprocessing:** stored data is cleaned and converted into a feature-engineered dataset.
3. **Demand forecasting model:** features produce a 7/30-day demand forecast.
4. **Inventory-risk module:** forecast, stock and reorder thresholds produce a stock alert.
5. **VADER sentiment module:** review text produces a sentiment score.
6. **Pricing recommendation module:** forecast, cost, risk and sentiment produce a constrained recommended price with an explanation.
7. **React dashboard and manager approval:** all outputs are displayed; the manager approves or rejects the price, and rejected or adjusted cases return for re-evaluation.

4.2 System Architecture

The architecture has four layers: (a) **Data sources:** product/pricing, sales, inventory and review CSV files; (b) **MySQL database:** central storage; (c) **Flask and ML layer:** REST API, data preprocessing, demand forecasting, inventory-risk, VADER sentiment and pricing recommendation modules; (d) **React dashboard:** visualisation and manager approval, with an approval loop back to the pricing module.

[Insert the System Architecture diagram from Slide 11 of the Phase II presentation here, together with a Level-0/Level-1 DFD if required by the department.]

5. Hardware and Software to be Used

Hardware: Standard laptop/desktop (minimum Intel i5 or equivalent, 8 GB RAM, 256 GB storage); GPU optional and needed only if an LSTM model is trained on larger data.

Software:

- **Frontend:** React, chart library for the dashboard
- **Backend:** Python, Flask (REST API)
- **Database:** MySQL
- **ML / Data:** Pandas, NumPy, scikit-learn, XGBoost, LightGBM, optional TensorFlow/PyTorch (LSTM)
- **NLP:** VADER sentiment analyser
- **Tools:** VS Code, Jupyter Notebook, Git/GitHub, Postman

- **Data:** Historical or simulated sales, inventory and review datasets (simulated data is labelled as such)

Limitations: Initial results may rely on simulated or limited public data, so accuracy will not match a production-scale system. VADER is lexicon-based and may miss sarcasm or multilingual text. Price elasticity is estimated from limited price variation. Competitor prices and live marketplace feeds are not included in the base scope.

6. Application and Future Scope

Applications: inventory planning and reorder alerts for small online sellers; data-driven discount and price decisions; product-demand analysis for niche categories; sentiment-aware merchandising.

Future scope:

- Live competitor-price tracking and marketplace API integration.
- Reinforcement-learning-based pricing for adaptive policies [3], [4].
- Pre-trained time-series foundation models for zero-shot forecasting [6].
- Transformer-based sentiment models and multilingual review analysis.
- Joint optimisation of pricing and inventory ordering decisions [5].
- LLM assistant for natural-language merchant queries and explanations.
- Multi-product, multi-store scaling and deployment on cloud infrastructure.

7. Project Timeline

Timeline in weeks across the semester (✓ = active).

Phase / Activity	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12	W13-14
Requirement analysis and dataset preparation	✓	✓	✓	✓			
Database design and Flask API setup	✓	✓					
Demand forecasting (baseline, XGBoost/LightGBM, evaluation)		✓					
Sentiment analysis and inventory-risk module		✓	✓	✓			
Constrained pricing recommendation		✓	✓	✓	✓		
React dashboard and manager approval flow			✓	✓	✓		
Testing (high-demand, low-demand, low-stock scenarios)							
Documentation, screenshots and final report							

8. Conclusion

The proposed system brings demand forecasting, inventory-risk detection, customer sentiment and pricing into a single workflow for small e-commerce sellers. Its distinguishing features are the combination of sales and review signals, price recommendations bounded by business constraints, explainable outputs, and a manager-approval step that keeps humans in control of every price change.

9. References

[1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785-794.

[2] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[3] K. Safonov, "Neural network approach to demand estimation and dynamic pricing in retail," arXiv:2412.00920, Dec. 2024.

[4] G. U. Bhargavi, M. Balakrishna, H. Shivani, B. Bramhani and D. Pavithra, "Dynamic price optimization for ecommerce platform," SciTePress, 2025, pp. 18-23, doi: 10.5220/0013907300004919.

[5] L. E. Brammer, "Dynamic pricing and inventory optimization in e-commerce using machine learning," M.Sc. dissertation, Universidade Catolica Portuguesa, Dec. 2025. [Online]. Available: https://repositorio.ucp.pt/bitstreams/49345403-96c3-4055-85b3-64ca611a9f05/download

[6] A. F. Ansari *et al.*, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024, arXiv:2403.07815.

[7] S. R. Divakarla, P. Subedi, K. S. and G. C. V., "Dynamic e-commerce pricing: Optimizing routes and forecasting demand with machine learning," in *Proc. 3rd Int. Conf. Futuristic Technology (INCOFT)*, vol. 3, SciTePress, 2025, doi: 10.5220/0013621300004664.

[8] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346-1364, 2022.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.

[10] A. V. den Boer, "Dynamic pricing and learning: Historical origins, current research, and new directions," *Surveys in Operations Research and Management Science*, vol. 20, no. 1, 2015.

[11] C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM)*, 2014.

[12] Flask Documentation. [Online]. Available: https://flask.palletsprojects.com

[13] React Documentation. [Online]. Available: https://react.dev

[14] MySQL Reference Manual. [Online]. Available: https://dev.mysql.com/doc/